

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width  
self.height  
self.walls  
self.food_positions  
self.opponents  
self.agent_pos
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The environment includes `self.opponents`, which introduces multiple independent entities that occupy the shared grid space and dynamically execute their own actions alongside the main agent.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is defined as the complete history of everything that an intelligent agent has ever perceived through its sensors up to a given point in time. An agent's choice of action at any instant can depend on this entire accumulated history.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method gathers and returns current observational data about the world (such as `agent_pos`, `opponent_positions`, and `smells_food`), functioning as the agent's sensory inputs from the environment.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The agent can see its own position, opponent positions, and local tile status, but it cannot see the global layout of all walls or the exact coordinates of all remaining food items (only their total count).

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

A performance measure must evaluate external behavior because an agent's success is defined by how well it achieves its goals in the world, whereas internal computational effort has no direct bearing on task accomplishment.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

It does not change the category—the environment remains Partially Observable—but it deepens the partial observability. By hiding the toxic traps, the agent has even less information about the true global state of the environment and cannot perceive hidden hazards until it encounters them.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding `smells_toxin` provides critical local feedback about hidden hazards (`self.toxic_traps`), allowing the agent to update its internal state regarding nearby dangers.

This helps the agent act with rationality because it can use this sensory information to choose actions that avoid negative utility (such as penalty points or damage from traps), thereby maximizing its expected performance measure over time rather than moving blindly into hazards.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic example from Lecture 01 is an agent that maximizes a poorly designed performance measure (such as earning points purely for cleaning up dirt) by repeatedly dumping dirt back onto the floor and cleaning it up again, or leaving dirt in one square to intentionally cycle back and forth to farm points.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING